HTML and Javascript Basics for 2D Game Development

1. The HTML Canvas:

The <canvas> element is an HTML element that allows developers to draw and render 2D graphics directly in a web page, making it useful for creating animations and video games.

- Drawing: It allows developers to draw lines, circles, rectangles, images, and text.
- Performance: It is designed to support efficient, repeated drawing, making it suitable for animations and interactive applications.
- Hardware acceleration: Modern browsers can use the GPU to accelerate certain canvas rendering operations, depending on the browser, device, and rendering workload.
- Game development: It can be used to create 2D games, sprites, animated characters, and game environments directly in the browser using JavaScript.

It is supported by: Google Chrome, Mozilla Firefox, Microsoft Edge, Safari, and Opera.

To create the canvas:

```
<canvas id="idcanvas" width="1200" height="1000" style = "border:black 1px solid;">
```

We use javascript to draw inside the canvas. To do so we will need to load the canvas and get its 2D context for 2D games:

```
var canvas = document.getElementById('idcanvas');
var ctx = canvas.getContext('2d');
```

a. **Drawing a Rectangle:** `ctx.fillRect(x, y, width, height);`
b. **Drawing a cicle:**
```
ctx.beginPath();
ctx.arc(x, y, radius, 0, 2 * Math.PI);
ctx.fill();
```
c. **Filling the form:** `ctx.fill(); ctx.fillRect();`
d. **Drawing only border:** `ctx.stroke(); ctx.strokeRect();`
e. **Deleting a form:** `ctx.clear();ctx.clearRect();`

f. Colouring in Canvas:

- `rgb(red, green, blue)`: each color component ranges from `0` to `255`.
- `rgba(red, green, blue, alpha)`: `alpha` controls opacity, from `0` (transparent) to `1` (opaque).

g. Drawing Image on Canvas:

To draw an image on the canvas element we use the constructor `Image()`.

```
image = new Image();

image.src = "image_path.png";
```

On position (x,y):

```
img.onload = function () {
    ctx.drawImage(img, x, y);
};
```

h. Saving and Restoring Context:

The canvas context (`ctx`) stores drawing settings such as colors, line widths, transparency, and transformations.

`ctx.save()`: Saves the current drawing settings.

`ctx.restore()`: Restores the most recently saved drawing settings.

These methods are useful when drawing multiple game objects with different styles or transformations.

i. Drawing Text:

Text can be displayed on the canvas using:

```
ctx.fillText(text, x, y)
ctx.strokeText(text, x, y)
ctx.font
ctx.textAlign
```

j. Transformations:

```
ctx.translate(x, y);
ctx.rotate(angle);
ctx.scale(x, y);
```

2. Animations:

Animation is essential in 2D games because characters, enemies, and other objects need to move over time.

The `requestAnimationFrame(callback)` function asks the browser to execute a function before the next screen repaint.
`setInterval(callback, delay)` is a built-in JavaScript Web API function provided by the browser. It repeatedly executes a function at a specified time interval.

3. Keyboard Interaction using Javascript:

A game needs to respond to player input. JavaScript can detect keyboard events using `keydown` and `keyup`.

```
document.addEventListener("keydown", function(event) {
        // event.key to check which key is down
}


document.addEventListener("keyup", function(event) {
        // event.key to check which key is up
}
```

4. Game Loop

Most games repeatedly perform three tasks:

- Update: Change the positions and states of game objects.
- Draw: Render the objects on the canvas.
- Repeat: Continue until the game stops.

So the general shape of a javascript 2D game looks like:

```
function gameLoop() {
    update();
    draw();

    requestAnimationFrame(gameLoop);
}

function update() {
    // Update game objects
}

function draw() {
    // Draw game objects
}

gameLoop();
```